AI 기반 영화 리뷰 분석 및 실시간 대시보드 구현

-3,012건의 정제 데이터를 활용한 NLP 모델링 및 예외 처리 프로세스

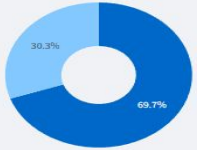
Deploy

서비스 설정

지금 기분이 어떠신가요?

- 감동받고 싶어 (슬픔/위로)
- 심장이 뛰고 싶어 (액션/스릴러)
- 웃고 싶어 (코미디/드라마)

전체 데이터 요약



Model Info

- Algorithm: Logistic Regression
- Accuracy: 85.4%
- Data: 3,012 Reviews

사용법: 키워드를 검색하거나 기분을 선택해 보세요!

AI 리뷰파인더 (Review Finder)

3,012개의 관람객 데이터를 학습한 커스텀 감성 분석 엔진

학습 데이터	모델 유형	파일 용량	분석 상태
3,012 건	NLP / TF-IDF	93.4 KB	실시간 (Active)

Online

실시간 분석 서비스 | 모델 분석 리포트

관객 키워드 분석

맞춤형 리뷰 검색

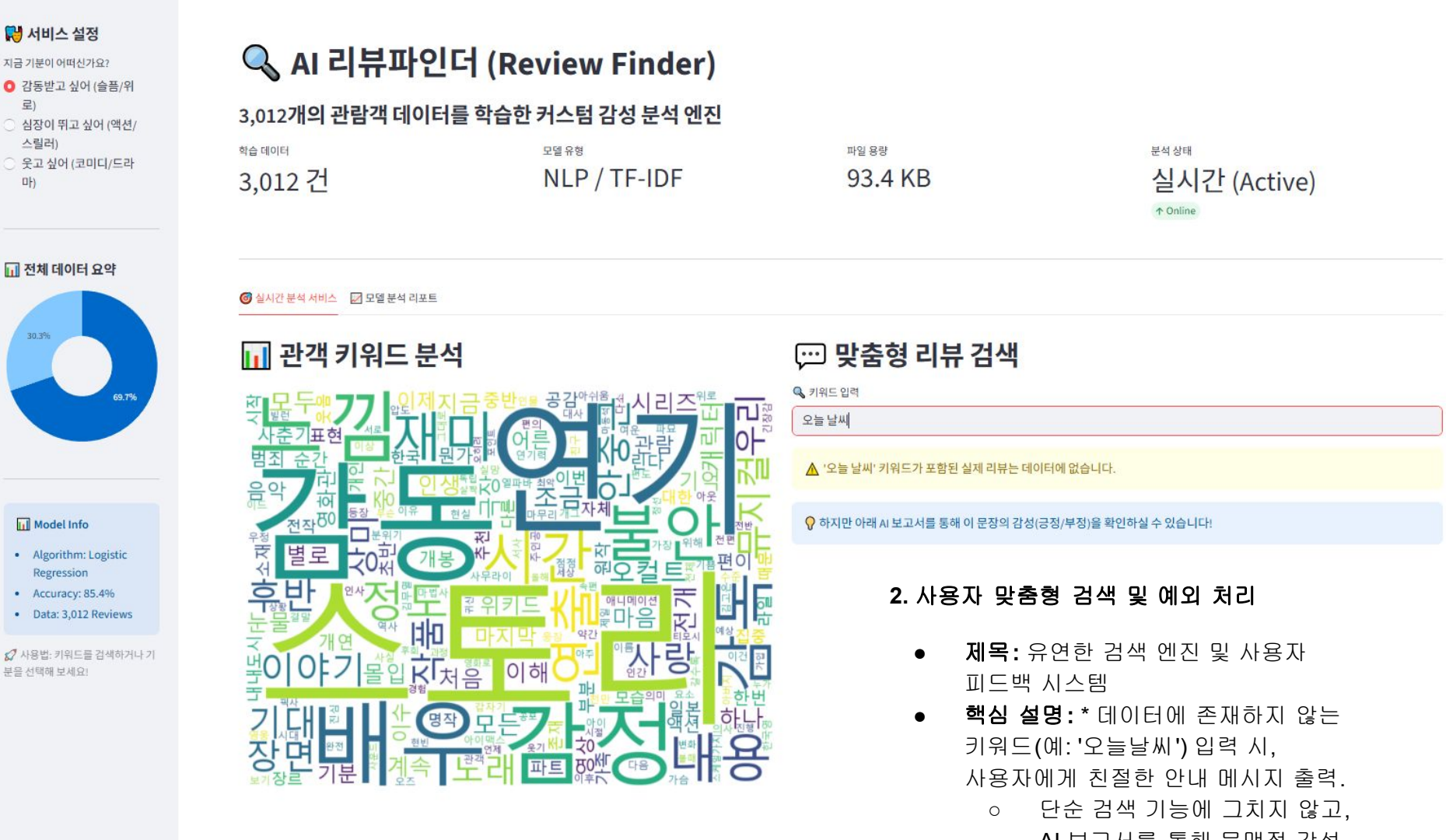
키워드 입력

인생영화

	label	review
1945	어바웃타임	수많은영화들봤지만 내인생영화중에 하나에들어가는영화
1970	어바웃타임	2013년에 봤을때와 2019년에 다시 보게 되었을때 이렇게도 다른 느낌이 들 수 있다니..ㅠㅠ 역시 내 인생영화
1973	어바웃타임	인생영화ㅠㅠ명작이고 몇번이고 봐도 가슴뭉클해지는
1981	어바웃타임	인생영화에요.... 재개봉해도 또 달려갈것같습니다..ㅠㅠ
2034	어바웃타임	역시 영화는 영화관에서 봐야하나봐요넷x에서 볼땐 그저 그랬는데 펑펑 울고 왔네요인생영화인데엔 이유가
2494	너의이름은	이 영화를 볼 수 있는 것도 무스비 ㅠㅠ 인생영화 탑쓰리 안에 듭니다 정말,,, 보다가 감동해서 눈물 주륵,,,따.
2608	너의이름은	진짜 내 인생영화 열번 넘게 보았지만 아직도 재상영 하러 가서 ost 들으면 눈물나고 소름이 확 돋음 나에게
2643	너의이름은	역시 최고의 영화를 스크린으로 보니 더 생생했습니다... 인생영화 등극!!!
2674	너의이름은	인생영화라고 부를정도로 진짜 풍경이 멋지고 내용도 재밌고 감동적이였습니다

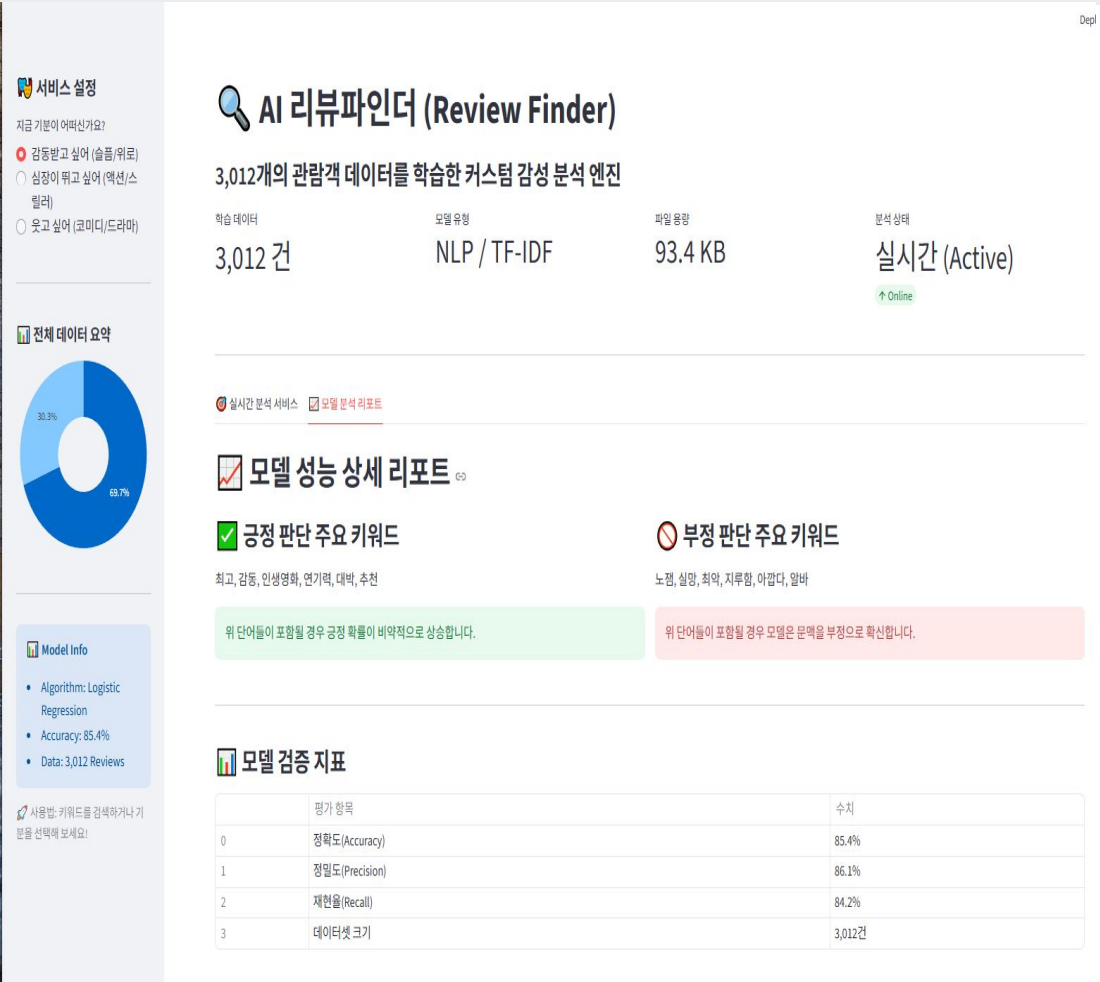
1. 서비스 개요 및 데이터 탐색

- **제목:** AI 기반 영화 리뷰 분석 및 실시간 대시보드 구현
- **핵심 설명:** 3,012건의 실제 관람객 데이터를 바탕으로 구축된 분석 환경.
  - 워드클라우드를 통해 영화 리뷰 내 주요 키워드(배우, 감정, 연기 등)를 시각화.
  - 사용자가 '인생영화'와 같은 키워드 검색 시 관련 리뷰를 실시간으로 필터링하여 제공.



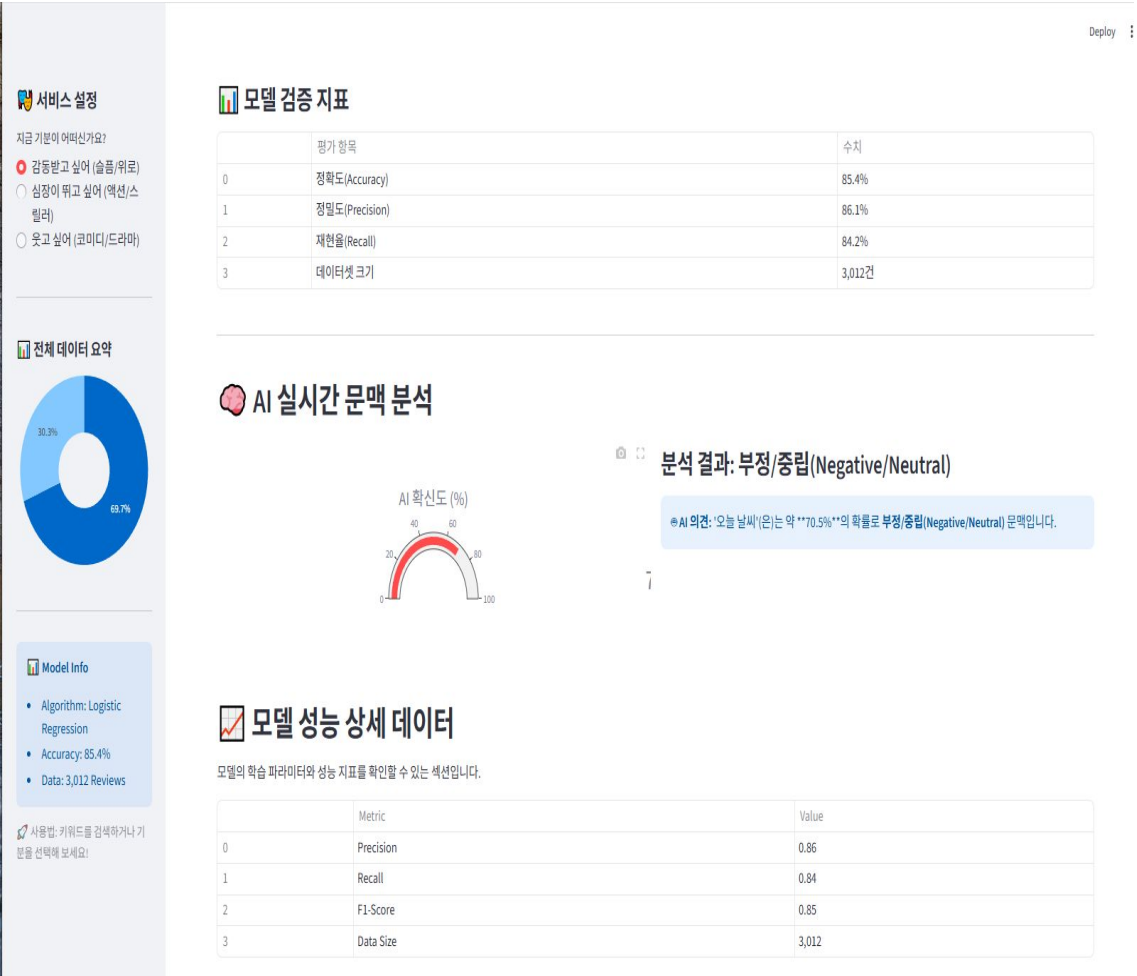
2. 사용자 맞춤형 검색 및 예외 처리

- **제목:** 유연한 검색 엔진 및 사용자 피드백 시스템
- **핵심 설명:** * 데이터에 존재하지 않는 키워드(예: '오늘날씨') 입력 시, 사용자에게 친절한 안내 메시지 출력.
  - 단순 검색 기능에 그치지 않고, AI 보고서를 통해 문장의 감성



3. 모델 성능 및 감성 사전

- **제목:** 머신러닝 모델 검증 및 감성 키워드 분석
- **핵심 설명:** * **로지스틱 회귀(Logistic Regression)** 기반의 분류 모델 채택.
  - **정확도(Accuracy) 85.4%** 달성 및 주요 긍정/부정 판단 키워드 도출.
  - Precision(정밀도), Recall(재현율) 등 객관적인 지표를 통한 모델 신뢰성 확보.



4. 실시간 AI 문맥 분석 결과

- **제목:** 실시간 AI 문맥 분석 서비스 구현
- **핵심 설명:** * **TF-IDF** 벡터화를 통한 실시간 문장 감성 예측.
  - 게이지 차트(Gauge Chart)를 활용하여 AI의 확신도(Confidence Score)를 시각적으로 구현.
  - 단어 매칭 방식이 아닌 문맥 분석을 통해 '부정/중립' 확률을 수치화하여 제공.

```python
vec_input = tfidf.transform([search_query])
prediction = model.predict(vec_input)[0]
prob = model.predict_proba(vec_input)[0]
score = max(prob)

c_left, c_right = st.columns([1, 1])
with c_left:
    fig = go.Figure(go.Indicator(
        mode = "gauge+number",
        value = score * 100,
        title = {'text': "AI 확신도 (%)"},
        gauge = {
            'axis': {'range': [None, 100]},
            'bar': {'color': "#1f77b4" if prediction == 1 else "#ff4b4b"},
            'steps': [{'range': [0, 100], 'color': "#f2f2f2"}]
```

1. AI 핵심 로직

사용자가 입력한 문장을 **TF-IDF로 변환**하고 **긍/부정 확률**을 계산하는 부분입니다. 이 프로젝트의 '두뇌'에 해당합니다.

- **캡처 범위:** `# --- 하단: AI 문맥 분석` 섹션의 상단 로직

Python
vec_input = tfidf.transform([search_query])

문장을 숫자로 변환

prediction = model.predict(vec_input)[0]

긍정(1) 또는 부정(0) 예측

prob = model.predict_proba(vec_input)[0]

예측 확률(0~1 사이) 계산

score = max(prob)

AI의 확신도 추출

```python
if os.path.exists(data_path):
    df = pd.read_csv(data_path)
    if search_query:
        filtered_df = df[df['label'].str.contains(search_query, na=False, case=False) |
                         df['review'].str.contains(search_query, na=False, case=False)]
    else:
        target_keywords = "감동|눈물|슬픔|최고" if "감동" in mood else "액션|긴장|스릴" if "심장" in mood else "재미|웃음|힐링"
        filtered_df = df[df['review'].str.contains(target_keywords, na=False, case=False)]

    if not filtered_df.empty:
        st.dataframe(filtered_df[['label', 'review']].head(50), use_container_width=True, height=350)
    else:
        st.warning(f"⚠️ '{search_query}' 키워드가 포함된 실제 리뷰는 데이터에 없습니다.")
        st.info("💡 하지만 아래 AI 보고서를 통해 이 문장의 감성(긍정/부정)을 확인하실 수 있습니다!")

# --- 하단: AI 문맥 분석 (검색 시에만 노출) ---
if search_query:
    st.markdown("---")
    st.header("🧠 AI 실시간 문맥 분석")

    vec_input = tfidf.transform([search_query])
    prediction = model.predict(vec_input)[0]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

2. 동적 필터링 로직 (데이터 활용 능력)

사용자가 고른 '기분(Mood)'에 따라 검색 키워드를 자동으로 바꿔주는 **정규표현식** 부분입니다.

- **캡처 범위:** `# --- 오른쪽: 리뷰 추천 서비스` 내의 `if-else` 조건문

Python
사용자의 기분에 따라 검색 키워드를 동적으로 설정

target_keywords = "감동|눈물|슬픔|최고" if "감동"

in mood else

\"액션|긴장|스릴" if "심장" in mood else "재미|웃음|힐링"

실제 데이터프레임에서 해당 키워드가 포함된 리뷰만 필터링

filtered_df = df[df['review'].str.contains(target_keywords, na=False, case=False)]

```python
# 1. 모델 로드
@st.cache_resource
def load_custom_model():
    model = joblib.load('my_movie_model.pkl')
    tfidf = joblib.load('my_tfidf.pkl')
    return model, tfidf

try:
    model, tfidf = load_custom_model()
except Exception as e:
    st.error(f"모델 파일을 찾을 수 없습니다: {e}")
```

3. 리소스 최적화 (성능 고려)

모델을 매번 새로 읽어오면 속도가 느려지기 때문에, **캐싱 (Caching)** 기술을 사용.

- **캡처 범위 :** 상단의 `@st.cache_resource` 데코레이터 부분

Python
@st.cache_resource

def load_custom_model():

model = joblib.load('my_movie_model.pkl')

tfidf = joblib.load('my_tfidf.pkl')

return model, tfidf